

GEOAIR

INTELLIGENCE PLATFORM

Master Project Plan · Version 3.0

Project	GeoAir Geopolitical Aviation Intelligence Platform
Author	Jack
Version	3.0 — Revised Master Plan
Timeline	18 months across 6 phases
Primary Stack	Python 3.11 (FastAPI) + Rust (PyO3)
Architecture	Layered monolith, fully containerised via Docker Compose

This document is the complete technical, strategic, and operational build guide for GeoAir. Update it as the project evolves.

CHAPTER 1

Project Vision, Goals & Success Criteria

1. Project Vision, Goals & Success Criteria

1.1 What is GeoAir?

GeoAir is a real-time geopolitical aviation intelligence platform. It continuously ingests news from hundreds of global sources, applies natural language processing to detect and score geopolitical events, geocodes those events to geographic coordinates, and correlates them with live flight data pulled from the OpenSky Network. The output is a unified, live dashboard showing — in real time — how events on the ground are affecting the skies above them.

In practical terms: if tensions escalate in a particular region, GeoAir detects the sentiment shift in news coverage, identifies the geographic focus, pulls all aircraft currently operating in that airspace, and generates a correlation alert with a confidence score. The dashboard displays affected flights on a live map overlaid with the event zone, and the news panel shows the articles that triggered it.

1.2 The Core Thesis — Why This Is Unique

Dozens of tools track flights. Dozens track news. None connect them intelligently. The unique engineering insight behind GeoAir is bidirectional correlation: geopolitical events inform route risk analysis, and conversely, anomalous flight behaviour — sudden mass rerouting, airspace avoidance — can serve as an early signal of geopolitical events before they appear in mainstream news coverage.

This is not an incremental improvement on existing tools. It is a genuinely new category of intelligence product. The closest commercial equivalents are FlightAware Insight and Cirium analytics, both enterprise products costing thousands of pounds per month. GeoAir builds a free, open version of this concept.

◆ NOTE

The correlation engine is what separates GeoAir from a flight tracker with a news ticker. Every design and architectural decision should serve the quality and accuracy of that correlation.

1.3 Target User Personas

Persona	Description	Primary Use
The Builder (you)	Developer building and iterating on the platform	Full access, debugging, data quality checks
Aviation Enthusiast	Pilot or aviation professional wanting route intelligence	Detailed flight data, airspace correlations, historical queries
Interested Observer	Non-technical, curious about world events and aviation	Dashboard view, alert notifications, map exploration
Future Public User	General public interested in geopolitics and travel safety	Clean dashboard, alert summaries, map interface

1.4 Success Criteria and Acceptance Tests

Each criterion is a concrete, binary test with a defined verification method. Never mark a phase complete until all Critical criteria for that phase pass.

#	Criterion	How to Verify	Phase	Priority
1	News articles ingested from 3+ sources continuously	Check DB: articles table has records with published_at within last 2 hours	1	Critical
2	Sentiment scores computed within 30 seconds of ingestion	Check processed flag and sentiment_score fields in DB	1	Critical
3	NLP domain accuracy > 80% on geopolitical test corpus	Manually label 50 articles, compare predicted vs actual sentiment	2	Critical
4	Live flight positions rendered on map, refreshed every 10s	Observe map, check timestamp in flight data panel	2	Critical
5	PyO3 Rust module callable from Python with correct types	Python unit test: import rust_core; assert type(result) == list	2	High
6	Geocoding accuracy > 85% on 50-article test set	Manual spot-check of 50 articles with known locations	3	High
7	Correlation alerts within 60s of triggering event	Insert test article, measure time to alert in DB	3	Critical
8	False positive rate < 25% on manually reviewed sample	Review 50 alerts, flag incorrect ones — improved from v2's 30% target	3	High
9	Dashboard loads in under 3 seconds on standard broadband	Chrome DevTools Network panel, from clean cache	4	Medium
10	System runs 7 days continuously without manual restart	Monitor uptime via healthcheck endpoint + UptimeRobot	6	Critical

1.5 The Extended Timeline Rationale

The 18-month timeline reflects the true scope of what you are building. You are simultaneously learning Rust, async Python, transformer-based NLP, time-series databases, spatial algorithms, and production DevOps. Each is a substantial undertaking in isolation. Compressing them creates shallow understanding and technical debt that will cost more to unwind than the time saved.

The 18 months are divided into 6 phases of roughly 10-12 weeks each. Every phase ends with a working, demonstrable product. Never be in a state where nothing runs. If a phase falls behind, cut scope within the phase rather than delaying the milestone — a working simpler version always beats a broken complex one.

Phase	Name	Weeks	Deliverable
1	Foundation & News Aggregator	1–10	Working news ingestion, NLP scoring, geocoding, basic browser UI
2	Flight Data Layer	11–20	Live flights on map, WebSocket updates, Rust ingestion pipeline
3	Correlation Engine v1	21–32	End-to-end correlation alerts, confidence scoring, map overlays
4	Hardening & Deployment	33–44	Cloud deployment, monitoring, 7-day stability, documentation
5	Correlation Engine v2 + React	45–56	Polygon matching, GDELT integration, React frontend migration
6	Production Hardening & Launch	57–72	14-day uptime, monitoring stack, public launch

CHAPTER 2

Full System Architecture

2. Full System Architecture

2.1 Architectural Overview

GeoAir is built as a layered monolith with clear internal boundaries. It is not a microservices architecture — that would add unnecessary deployment complexity for a solo project. Instead, it is a well-structured single application with logically separate modules, running as a set of Docker containers orchestrated by Docker Compose. This gives most of the organisational benefits of microservices without the operational overhead.

Layer	Technology	Responsibility	Language
Data Sources	External APIs	Provide raw news and flight data	N/A (external)
Ingestion	Rust + tokio	Fetch, parse, validate, store at volume with zero-copy parsing	Rust
Processing	Python + spaCy + HuggingFace	NLP pipeline, geocoding, correlation logic, domain model scoring	Python
Serving	FastAPI + uvicorn	REST API, WebSocket push, JWT authentication	Python
Persistence	PostgreSQL + TimescaleDB	All data storage, time-series queries, spatial index	SQL
Frontend	HTML/CSS/JS → React 18	Dashboard, live map, alert panel, analytics	JavaScript

2.2 End-to-End Data Flow

Data flows in one primary direction: from external sources through ingestion, processing, and serving, out to the dashboard. The correlation engine is a cross-cutting concern that reads from both the news and flight data streams simultaneously.

Step	Action	Component	Output	Timing
1	Scheduled trigger fires	Rust scheduler	HTTP requests dispatched	Every 60s / 10s
2	HTTP response parsed and deduplicated	Rust parser	Raw records	Immediate
3	Raw records written to staging tables	Rust + sqlx	DB rows in articles_raw / flights_raw	Immediate
4	NLP pipeline reads unprocessed articles	Python APScheduler	Entity + sentiment annotations with domain calibration	Every 30s

Step	Action	Component	Output	Timing
5	Entities geocoded to lat/lon coordinates	Python geopy + cache	Coordinate pairs with confidence score	Every 30s
6	Correlation engine checks events vs flights	Python correlation	CorrelationResult objects with versioned algorithm	Every 45s
7	High-confidence results written to DB	Python SQLAlchemy	DB rows in correlations with algorithm_version tag	Every 45s
8	WebSocket broadcasts new alerts to clients	FastAPI WebSocket	JSON messages pushed to all connected clients	Immediate
9	Frontend receives update, re-renders	JavaScript	Visual map + alert update without page refresh	Immediate

2.3 Service Communication Patterns

- Rust → Python: Via the shared PostgreSQL database. Rust writes raw data; Python reads it. No direct process-to-process communication is needed for normal operation.
- Python PyO3 calls: For batch ingestion tasks triggered from Python, the Rust module is imported and called directly as a Python function. Returns data as Python-native types.
- Python → Frontend: REST API for page loads and initial data. WebSocket for live updates. The frontend never polls — it receives push notifications.
- Database → All: PostgreSQL is the central shared state. All components that need to communicate asynchronously do so through the database.

2.4 Scalability Considerations

GeoAir v1.0 is designed for a single server. The architecture accommodates future scaling with minimal refactoring:

- Horizontal ingestion scaling: Multiple Rust ingestion workers can run in parallel without coordination, since PostgreSQL handles deduplication via UNIQUE constraints on article URLs.
- Queue-based decoupling: If ingestion volume grows, replace the direct DB write with Redis Streams or PostgreSQL LISTEN/NOTIFY between ingestion and processing.
- NLP batch processing: Move sentiment inference to a separate worker process with GPU access if throughput demands it.
- Read replicas: The serving layer can point to a PostgreSQL read replica for query traffic, leaving the primary for writes only.

CHAPTER 3

Technology Stack — Every Choice Justified

3. Technology Stack — Every Choice Justified

3.1 Why Python + Rust Together

The combination is intentional and well-established in production systems. Python's ecosystem for NLP, data processing, and web frameworks is unmatched. Rust provides memory safety, zero-cost async concurrency, and performance for the ingestion pipeline. The PyO3 bridge lets both coexist cleanly with minimal friction.

This is not an experimental architecture. Dropbox, Cloudflare, Discord, and Mozilla all use Rust for performance-critical components while maintaining Python or other languages for application logic.

◆ NOTE

Rust is used for exactly one component in v1.0: the ingestion pipeline. If Rust work falls behind schedule, ingestion can be temporarily replaced with a pure Python implementation. The rest of the system is completely unaffected. Never let Rust block progress on the rest of the platform.

3.2 NLP Stack — Improved Model Selection

The original v2.0 plan recommended `cardiffnlp/twitter-roberta-base-sentiment-latest`. This is a known domain mismatch risk: the model was trained on tweets, which differ structurally from Reuters wire copy, GDELT event descriptions, and AP dispatches. Version 3.0 introduces a tiered approach:

Model	Strengths	Weakness	Use When
distilbert-sst-2	Fast (CPU ~80ms), small footprint	Binary only (pos/neg), low nuance	Dev / rapid iteration
ProsusAI/finbert	Trained on financial/geopolitical text, 3-class output	Slower than distilbert	Phase 2 onwards
DeBERTa-v3-base (fine-tuned)	SOTA accuracy, best for geopolitical domain after fine-tuning	1-2s per article on CPU	Phase 5 production

◆ TIP

Start with ProsusAI/finbert in Phase 1 instead of the Twitter-trained model. It was trained on financial and geopolitical news, which matches the GeoAir domain far more closely and will produce fewer false positives from the outset.

3.3 Rust Crates — Full List

Crate	Role	Why This One
tokio	Async runtime	Industry-standard. Handles concurrent HTTP requests, timers, and tasks without blocking threads. Essential for I/O-heavy ingestion.
reqwest	HTTP client	Async HTTP built on tokio. Handles connection pooling, redirects, TLS, and timeouts. The standard choice.
serde + serde_json	Serialisation	Zero-cost derive macros. Deserialises API responses directly into typed Rust structs at compile time.
pyo3	Python bridge	Exposes Rust code as a Python extension module. Handles type conversion natively. The critical interop layer.
sqlx	Async DB driver	Compile-time verified SQL queries. Async PostgreSQL support. Prevents entire classes of SQL errors at compile time.
chrono	Date/time	Handles all timestamp parsing across varied news API formats.
thiserror	Error types	Reduces boilerplate for custom error types. Makes error handling readable and idiomatic.
tokio-retry	Retry logic	NEW: Exponential backoff retry for HTTP requests. Prevents transient API failures from losing data.
tracing	Structured logs	NEW: Structured logging with spans. Replaces println!. Integrates with the monitoring stack in Phase 6.

3.4 Python Packages — Full List

Package	Role	Why This One / Notes
fastapi	Web framework	Modern async Python web framework. Auto-generates OpenAPI docs, native async/await, WebSocket support.
uvicorn + gunicorn	ASGI server	uvicorn for dev. gunicorn multi-worker wrapper in production for fault isolation.
sqlalchemy[asyncio]	ORM	Python class models mapping to DB tables. Async via asyncpg. Schema evolution via Alembic.
asyncpg	DB driver	Fastest async PostgreSQL driver. Used under SQLAlchemy.
alembic	Migrations	Manages schema changes as versioned scripts. Never alter schema without a migration.
spacy (en_core_web_lg)	NLP — entities	Industrial NER for GPE/ORG/PERSON entities. The lg model is required for acceptable accuracy.
transformers + torch	NLP — sentiment	HuggingFace + PyTorch. Start with ProsusAI/finbert. CPU-only installation sufficient.
geopy + redis	Geocoding + cache	geopy for Nominatim. NEW: Redis cache layer to handle volume at 1s/request rate limit.
shapely	Geometry	Point-in-polygon tests, distance calculations, FIR boundary intersection.
apscheduler	Scheduling	Background job management. All ingestion, NLP, and correlation runs scheduled here.
slowapi	Rate limiting	NEW: Per-IP request rate limiting for the public API endpoints.

Package	Role	Why This One / Notes
prometheus-fastapi-instrumentator	Metrics	NEW: Auto-instruments FastAPI endpoints with Prometheus metrics for Phase 6 monitoring.
pytest + pytest-asyncio	Testing	Standard test framework. Required before marking any phase milestone complete.

CHAPTER 4

Development Environment Setup

4. Development Environment Setup (Explicit Steps)

◆ NOTE

All commands assume macOS on Apple Silicon (M1/M2/M3/M4/M5). All tools listed have native ARM64 support. Do not install x86 versions via Rosetta — they cause subtle compatibility issues with Python native extensions.

4.1 System Prerequisites

Tool	Install Command	Verify
Homebrew	<code>/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"</code>	<code>brew --version</code> — should print 4.x or higher
Git identity	<code>brew install git && git config --global user.name "Jack" && git config --global user.email "your@email.com"</code>	<code>git --version</code>
pyenv + Python 3.11	<code>brew install pyenv && pyenv install 3.11.9 && pyenv global 3.11.9</code>	<code>python --version</code> — must print Python 3.11.9
Rust via rustup	<code>curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs sh</code>	<code>rustc --version</code> — should print rustc 1.77 or higher
PostgreSQL 16 + TimescaleDB	<code>brew install postgresql@16 timescaledb-tools && brew services start postgresql@16</code>	<code>psql --version</code> — should print 16.x
Redis (geocoding cache)	<code>brew install redis && brew services start redis</code>	<code>redis-cli ping</code> — should return PONG
maturin (PyO3 build)	<code>pip install maturin</code>	<code>maturin --version</code> — should print 1.x
Node.js (frontend)	<code>brew install node</code>	<code>node --version</code> — should print v20 or higher
Docker Desktop	Download from docker.com — install the Apple Silicon (ARM) version	<code>docker --version</code> — should print 24 or higher

4.2 Database Setup

1. Create the geoair database: `createdb geoair`
2. Enable TimescaleDB: `psql geoair -c "CREATE EXTENSION IF NOT EXISTS timescaledb;"`

3. Enable PostGIS (for spatial queries in Phase 5): `psql geoair -c "CREATE EXTENSION IF NOT EXISTS postgis;"`
4. Create a dedicated DB user: `createuser geoair_user` with password, grant privileges.
5. Verify: `psql geoair -c "SELECT extname FROM pg_extension;"` — must list timescaledb and postgis.

◆ WARNING

PostGIS enables native spatial queries in Phase 5 (ST_Within, ST_DWithin) which are significantly faster than pure Python shapely calculations on large flight datasets. Install it now so the schema can support it from the start.

4.3 Project Structure

Create the following directory structure. This is the canonical layout — do not deviate from it as the phases build on it incrementally.

Directory / File	Purpose
rust_core/src/	Rust ingestion pipeline source
rust_core/src/ingestion/	News and flight fetchers
rust_core/src/db.rs	Database write functions
rust_core/src/errors.rs	Custom error types
backend/api/routes/	FastAPI route handlers
backend/nlp/	NLP pipeline, sentiment, entity extraction
backend/geo/	Geocoding, spatial calculations
backend/correlation/	Correlation engine (versioned)
backend/db/	SQLAlchemy models, migrations, queries
backend/scheduler.py	APScheduler job definitions
frontend/js/	Vanilla JS (Phases 1-4)
frontend-react/	React app (Phase 5+)
tests/python/	Python test suite
tests/fixtures/	Cached API responses for offline testing
docs/	Architecture, API reference, incident log
.env.example	Template — never commit .env
docker-compose.yml	Service orchestration

CHAPTER 5

Data Sources & API Integration

5. Data Sources & API Integration

Source	Free Tier	Rate Limit	Best For	Notes
NewsAPI	Yes	100 req/day	English headlines, breaking news	Upgrade to paid for production. Cache all responses as JSON fixtures.
GDELT Project	Yes	Unlimited	Historical, non-English, Goldstein scale	15-minute bulk CSV. Primary source from Phase 2 onwards.
OpenSky Network	Yes (registered)	4,000 req/day	Live ADS-B positions globally	Register free account. Reduce polling to 30s globally; 10s for active event zones only.
ADS-B Exchange	Yes (limited)	Varies	Supplementary / OpenSky fallback	No filtering by authority — useful for unredacted military flights.
OurAirports	Yes	N/A (static)	FIR boundary polygons	Download once, import to DB. Essential for Phase 5 polygon matching.
RSS Feeds (BBC/Reuters/AP/Al Jazeera)	Yes	Self-managed	Real-time breaking news with no API key	Parse with feedparser. No quota. Add in Phase 2.

◆ WARNING

OpenSky anonymous access is 400 requests/day. Registered accounts get 4,000/day. At 10-second global polling the system needs 8,640/day — over the limit. Solution: reduce global polling to 30 seconds. Increase to 10 seconds dynamically only for bounding boxes that contain an active high-severity correlation event.

5.1 Geocoding Architecture — Revised for Volume

The v2.0 plan used a simple Nominatim wrapper with LRU cache. This breaks at production volume. Once processing hundreds of articles per hour, the 1 request/second rate limit becomes a hard bottleneck. Version 3.0 uses a tiered geocoding strategy:

Tier	Source	Latency	Use Case
1	LOCATION_OVERRIDES dict (in-memory)	< 1ms	Known high-frequency entities: "Ukraine", "Gaza", "Taiwan", "Kashmir"

Tier	Source	Latency	Use Case
2	Redis cache (TTL = 30 days)	< 5ms	Any place name geocoded in the last 30 days
3	Nominatim (rate-limited)	~1s	Novel place names not seen before
4	Country-level fallback	< 1ms	When city-level geocoding fails — use bounding box of parent country

◆ NOTE

Only geocode entities labelled GPE (geopolitical entity) by spaCy — never generic LOC tags. "The region" and "the conflict zone" will return garbage geocoding results and inflate false positive rates. GPE-only filtering eliminates the majority of bad geocodes before they reach Nominatim.

CHAPTER 6

Deep Dive: Rust Ingestion Pipeline

6. Deep Dive: Rust Ingestion Pipeline

The ingestion pipeline is the highest-frequency, most I/O-intensive part of the system. At 30-second flight polling intervals and continuous news fetching, it makes hundreds of HTTP requests per hour and parses large JSON payloads. Rust's async runtime (tokio) handles this with minimal memory overhead and zero garbage collection pauses.

◆ NOTE

An async function in Rust returns a Future — an object representing a computation that has not finished yet. The tokio runtime polls these futures, advancing them when their I/O completes. Unlike threads, many thousands of concurrent async tasks can run on a small number of OS threads.

6.1 Key Patterns

Pattern	Description
<code>async fn / .await</code>	Define async functions identical to Python's <code>async def / await</code> syntax
<code>tokio::join!</code>	Run multiple futures concurrently — like Python's <code>asyncio.gather()</code>
<code>tokio::spawn</code>	Spawn fire-and-forget background tasks — e.g. the ingestion loop
<code>? operator</code>	Propagates errors up the call stack automatically — essential in ingestion
<code>#[derive(Error, Debug)]</code>	thiserror macro for custom error types — keeps ingestion errors structured
<code>tokio_retry::Retry</code>	NEW: Exponential backoff retry wrapper for all HTTP requests
<code>tracing::info! / warn!</code>	NEW: Structured log events — replaces <code>println!</code> throughout

6.2 Error Handling & Resilience

The v2.0 plan underspecified error handling in the ingestion layer. Every HTTP request must have a defined failure mode:

- Transient failures (5xx, timeout, network error): retry 3 times with exponential backoff (1s, 2s, 4s) plus random jitter (0–500ms). Log at WARN level.
- Rate limit (429): read the Retry-After header if present, otherwise back off for 60 seconds. Log at INFO level.
- Authentication failure (401/403): do not retry. Log at ERROR level and disable that source until the next startup.
- Malformed JSON (serde parse error): store the raw response to `tests/fixtures/failed/` for debugging. Log at WARN. Never panic.
- Database write failure: retry 3 times with 500ms delay. If all retries fail, write to a local dead-letter file and attempt DB re-insertion on the next ingestion cycle.

CHAPTER 7

Deep Dive: PyO3 Bridge

7. Deep Dive: PyO3 Bridge

PyO3 compiles Rust code into a shared library (.so on macOS/Linux) that conforms to the Python C extension API. When Python imports your Rust module, it loads this .so file exactly like any other C extension — NumPy works the same way.

◆ NOTE

PyO3 functions must be synchronous from Python's perspective, even if they call async Rust internally. Inside each `#[pyfunction]`, create a tokio runtime with `tokio::runtime::Runtime::new()` and use `rt.block_on()` to drive the async code. This is the standard pattern and is safe and correct.

7.1 Type Conversion Reference

Rust Type	Python Type	Notes
String	str	Automatic via PyO3
f64 / f32	float	Automatic
i64 / i32	int	Automatic
bool	bool	Automatic
Vec<T>	list	Automatic if T is a convertible type
Option<T>	T or None	Automatic — None maps to Python None
HashMap<String, V>	dict	Automatic if V is convertible
PyList	list	Manual construction — needed for heterogeneous lists
PyDict	dict	Manual construction — use <code>dict.set_item()</code>

CHAPTER 8

Deep Dive: NLP & Sentiment Engine

8. Deep Dive: NLP & Sentiment Engine

Modern sentiment analysis uses transformer neural networks. A pre-trained model has learned the relationship between word patterns and sentiment from hundreds of millions of examples. The key challenge for GeoAir is domain adaptation: general-purpose sentiment models systematically misclassify geopolitical language.

◆ CRITICAL

Domain mismatch is the #1 source of correlation false positives. "Russia launches new initiative" is positive sentiment in a general model. "Airspace closure declared" is neutral in a Twitter-trained model. ProsusAI/finbert is the recommended starting point because it was trained on domain-adjacent text. Plan to build a labelled evaluation dataset of 200 geopolitical articles (100 with genuine risk, 100 without) by Week 12, and use it to validate every model switch.

8.1 The Seven-Step NLP Pipeline (Revised)

#	Step	Function	Output	Failure Behaviour
1	Language detection	detect_language()	ISO 639-1 code	Default to "en". Skip non-English in Phase 1; add multilingual in Phase 5.
2	Relevance filter	is_geopolitically_relevant()	Boolean	Skip article if False — saves NLP compute on unrelated articles.
3	Entity extraction	extract_geopolitical_entities()	List[ExtractedEntity]	Store empty list, continue pipeline.
4	Primary location	get_primary_location()	GPE string or None	Store None. Engine skips article with no location.
5	Sentiment scoring	score_sentiment()	(score, label, confidence)	Store score=0.0, label="neutral". Requires confidence > 0.6 threshold.
6	Geocoding	geocode_location()	(lat, lon, tier_used)	Store None. Engine skips article. Log tier for calibration metrics.
7	Store results	update_article()	DB write	Retry 3x, then mark as failed with error_reason field.

8.2 Evaluation Dataset — Build This Early

The single most important addition in v3.0: build a labelled evaluation dataset before you start tuning thresholds. Without it, you are calibrating blind.

Dataset Split	Content	Size
True Positives	Real geopolitical events with confirmed aviation impact (airspace closures, mass diversions)	100 articles
True Negatives	Geopolitical news with zero aviation relevance (diplomatic talks, elections, trade policy)	100 articles
Hard Negatives	Geopolitical events near aviation but no actual impact — these cause most false positives	50 articles

Populate the dataset using historical news from the events in Chapter 14.3. Store it in tests/fixtures/evaluation_corpus/ as JSON. Run it against the model every time you change a threshold, model, or confidence weight. Track precision, recall, and F1 — not just false positive rate.

CHAPTER 9

Deep Dive: Correlation Engine

9. Deep Dive: Correlation Engine

The correlation engine answers: given a geopolitical event in unstructured text, which aircraft are at risk, how significant is that risk, and how confident are we? Getting this right requires combining spatial reasoning, temporal analysis, and a probabilistic confidence model.

◆ NOTE

There is no existing library or tutorial for this. You are designing an original algorithm. Start simple (bounding circle + sentiment threshold), measure accuracy on real events, and iterate. Version the algorithm explicitly so you can compare performance on historical data. Never delete a version — tag correlations with `algorithm_version` so older and newer results can be compared.

9.1 Algorithm Version Roadmap

Version	Phase	Method	New This Version	Target FP Rate
v1	Phase 3	Bounding circle (haversine radius) + sentiment threshold	Baseline correlation, confidence scoring	< 30%
v2	Phase 5	FIR polygon matching via OurAirports + PostGIS	Goldstein scale from GDELT, source credibility weighting	< 20%
v3	Phase 6	v2 + historical calibration, adaptive thresholds per region	Per-region baseline volatility, recency decay curve	< 15%

9.2 Confidence Scoring Formula (v1)

The confidence score is a weighted sum of five factors, producing a value from 0.0 to 1.0. Note the addition of a model confidence factor versus v2.0 — this penalises correlations where the NLP model itself was uncertain:

Factor	Weight	Description	Range
Sentiment severity	35%	How negative is the article — <code>abs(score)</code> . Requires finbert confidence > 0.6	0.0 to 1.0
Flight exposure	25%	Number of aircraft in zone, capped at 50	0.0 to 1.0
Source credibility	20%	Reuters/AP/BBC score 1.0; unknown domains score 0.4	0.0 to 1.0
Event recency	10%	Articles older than 12h decay linearly toward 0	0.0 to 1.0

Factor	Weight	Description	Range
Model confidence (NEW)	10%	The probability score from the NLP classifier. Low-confidence predictions should lower overall correlation confidence.	0.0 to 1.0

◆ WARNING

These weights are initial estimates to be validated against the evaluation corpus (Chapter 8.2). Before deploying v1, run the confidence formula over all 250 labelled articles and measure precision/recall at the alert thresholds below. Adjust weights if precision < 0.70 at the HIGH threshold.

9.3 Alert Level Classification

Level	Confidence Threshold	Severity Threshold	Action
CRITICAL	≥ 0.75	≥ 0.70	Immediate WebSocket broadcast, map highlight, email notification
HIGH	≥ 0.55	Any	WebSocket broadcast, alert panel card
MEDIUM	≥ 0.35	Any	Alert panel card only — no broadcast
LOW	< 0.35	Any	DB only — not displayed in default view

CHAPTER 10

Database Design

10. Database Design

Table	Type	Write Frequency	Primary Query Pattern
articles	Relational	~60s	Unprocessed queue; by sentiment, region, date
flight_positions	TimescaleDB hypertable	~30s	Latest per aircraft; all in bounding box in last 30s
correlations	Relational	~45s	Active unacknowledged; by region, date, algorithm_version
users	Relational	Rare	By username/email for authentication
airspace_regions	Relational (PostGIS)	Static	ST_Within / ST_Intersects point-in-FIR queries
user_watchlist	Relational	On demand	By user_id for personalised dashboard
nlp_evaluation_results	Relational (NEW)	On model change	Track precision/recall/F1 per model version over time

10.1 Critical Indexes

The following indexes must be created for acceptable query performance. Without them, the correlation engine's per-45s flight queries will time out as the flight_positions table grows past a few million rows.

Index Name	Columns / Condition	Powers
idx_articles_unprocessed	fetches_at WHERE processed = FALSE	NLP pipeline queue
idx_articles_geo	(lat, lon) WHERE lat IS NOT NULL	Spatial article lookup
idx_articles_sentiment	sentiment_score	Negative article filtering for correlation
idx_flights_icao_time	(icao24, time DESC)	Per-aircraft position history
idx_flights_spatial (PostGIS)	GIST index on geography column	ST_DWithin radius queries in v2
idx_correlations_active	detected_at WHERE acknowledged = FALSE	Active alerts API
idx_correlations_version	(algorithm_version, detected_at)	Version comparison queries in Phase 5

10.2 Data Retention

Flight positions grow at approximately 500MB–1GB per day at global scale. Set the retention policy in Phase 1, not later. TimescaleDB drops old chunks automatically — no expensive DELETE operations.

- Flight positions: 90-day retention. `SELECT add_retention_policy('flight_positions', INTERVAL '90 days');`
- Raw articles: 180-day retention. Processed articles (with sentiment scores) kept indefinitely for historical analysis.
- Correlations: kept indefinitely — they are the core analytical output.
- Monitor disk weekly: `SELECT pg_database_size('geoair');`

CHAPTER 11

FastAPI Backend

11. FastAPI Backend

11.1 API Endpoint Reference

Method	Auth	Endpoint	Description	Phase
GET	Public	/api/news/latest	Last 50 processed articles, sorted by published_at desc	1
GET	Public	/api/news/sentiment	Sentiment scores over time for a region	1
GET	Public	/api/news/map-data	Geocoded articles only, for map overlay	1
GET	Public	/api/news/search	Full-text search across article titles and content	3
GET	Public	/api/flights/live	All flight positions from last 30s as GeoJSON	2
GET	Public	/api/flights/region	Flights in bounding box	2
GET	Public	/api/flights/{icao24}/history	Position history for aircraft (last 6 hours)	2
GET	Public	/api/correlations	All correlations, filterable by level, date, algorithm_version	3
GET	Public	/api/correlations/active	Unacknowledged correlations by confidence	3
POST	Required	/api/alerts/{id}/acknowledge	Mark alert as acknowledged	3
POST	Public	/api/auth/register	Create new user account	4
POST	Public	/api/auth/token	Login — returns JWT access token	4
GET	Public	/api/health	System health: DB status, ingestion lag, article count	1
GET	Public	/metrics	Prometheus metrics endpoint	6
WS	Public	/ws/live	WebSocket: broadcasts alerts, flight updates, ingestion status	2

11.2 Background Task Schedule

Job	Interval	max_instances	Notes
run_news_ingestion	60 seconds	1	Calls rust_core.fetch_news(). Stores raw articles.
run_rss_ingestion	5 minutes	1	feedparser: BBC, Reuters, AP, Al Jazeera. Add Phase 2.
run_flight_ingestion	30 seconds	10 seconds if active alert	1
process_pending_articles	30 seconds	1	NLP pipeline: language detect → filter → entities → sentiment → geocode.
run_correlation_pass	45 seconds	1	Correlation engine. Stores results with algorithm_version tag.
run_gdelt_ingestion	15 minutes	1	GDELT bulk CSV download and parse. Add Phase 5.
run_nlp_evaluation	On demand	1	Runs evaluation corpus against current model. Log precision/recall.

CHAPTER 12

Frontend & UI Design

12. Frontend & UI Design

Phase	Technology	Rationale
1–2	Vanilla JS + Leaflet.js	No build step, no bundler overhead. Focus stays on data, not framework.
3–4	Vanilla JS + Chart.js added	Add sparklines and analytics charts. State still simple enough for vanilla.
5+	React 18 + react-leaflet	Migrate when state management becomes complex. React.memo and useMemo essential for WebSocket performance with 10s flight updates.

12.1 Dashboard Layout

Panel	Width	Contents
Top navigation bar	Full width	Logo, global sentiment indicator, active alert count, last updated time, WebSocket connection status
Left — News feed	25%	Article cards with sentiment badges, colour-coded by score. Filter bar (region, severity, source). Infinite scroll.
Centre — Live map	50%	CartoDB Dark Matter tiles. Aircraft icons rotated to heading, event zone overlays, geocoded article pins.
Right — Alerts	25%	Alert cards sorted by confidence. Sentiment sparklines per region. Daily statistics. Acknowledge buttons.

12.2 React Migration — Scope Warning

◆ WARNING

The React migration is scoped as 4 weeks in v2.0. This is optimistic. A real-time WebSocket app with Leaflet — migrated to React + react-leaflet, with proper state management (useReducer) for high-frequency flight updates — is closer to 8 weeks of focused work. Phase 5 in v3.0 allocates 10 weeks to Phase 5 content for this reason. Do not rush the migration. Run both frontends in parallel during the transition and only switch production after every feature is verified to work identically.

CHAPTER 13

Extended 18-Month Phase Build Plan

13. Extended 18-Month Phase Build Plan

◆ NOTE

Every phase ends with a working, demonstrable product. Never be in a state where nothing runs. If a phase falls behind, cut scope within the phase — never delay the milestone. A working simpler version beats a broken complex one every time.

PHASE 1 · FOUNDATION & NEWS AGGREGATOR

Weeks 1–10

✓ Working news ingestion, NLP pipeline, geocoding, basic browser UI — all continuously running

MILESTONE	Articles flowing from 3+ sources into DB continuously, with deduplication	Week 4
------------------	---	---------------

MILESTONE	Sentiment-scored articles with finbert displayed in browser with working filter	Week 8
------------------	---	---------------

MILESTONE	Geocoded article locations visible as pins on a live Leaflet map	Week 10
------------------	--	----------------

WEEK 1	Create project root, initialise Git, create full directory structure from Chapter 4.
	Complete all environment setup steps from Chapter 4.1 — verify each tool before proceeding.
	Create .env.example and .env, populate with NewsAPI key from newsapi.org (free tier).
	Write initial DB schema SQL: articles, flight_positions, correlations, users, nlp_evaluation_results.
	Initialise Alembic: alembic init migrations, configure alembic.ini, run alembic upgrade head.
	Create backend/main.py: minimal FastAPI app returning {"status": "ok"} at GET /. Verify in browser.

WEEK 2	Create backend/db/session.py: SQLAlchemy async engine, session factory, get_db dependency.
	Create backend/db/models.py: SQLAlchemy ORM model for Article. Verify it maps correctly.
	Write backend/db/queries.py: insert_article(), get_unprocessed_articles(), mark_article_processed().
	Test DB writes manually: Python REPL → create test Article → verify in psql.
	Implement GET /api/news/latest: returns last 50 articles from DB as JSON.
	Write tests/python/test_api.py: verify /api/news/latest returns 200 with correct structure.

WEEK 3	Implement backend/nlp/entities.py: load spaCy en_core_web_lg at module level (load once — it's slow).
---------------	---

	Write <code>extract_geopolitical_entities(text) → List[ExtractedEntity]</code> using <code>doc.ents</code>, GPE filter only. Write <code>get_primary_location(entities) → Optional[str]</code>: most-mentioned GPE entity. Test interactively: paste a Reuters conflict article, verify correct country extraction. Write <code>tests/python/test_nlp.py</code>: entity extraction on 5 known articles with expected GPE outputs. Document any entities that fail to extract — first entries in your calibration log.
WEEK 4	Implement <code>backend/nlp/sentiment.py</code>: load <code>ProsusAI/finbert</code> via <code>HuggingFace</code> pipeline. First run downloads ~400MB. Write <code>score_sentiment(text) → tuple[float, str, float]</code> returning score, label, and model confidence. Write <code>is_geopolitically_relevant(text, threshold=-0.3)</code> as a pre-filter — saves inference compute. Test on 10 manually chosen articles: 4 clearly negative conflict, 3 neutral diplomatic, 3 positive. Add sentiment tests: known-negative conflict articles must score below -0.3 with confidence > 0.6. Record inference time per article on your hardware — determines batch size for 30s scheduler run.
WEEK 5	Install Redis and configure the geocoding cache layer from Chapter 5.1. Implement <code>backend/geo/geocoder.py</code>: tiered geocoding — in-memory overrides → Redis cache → Nominatim → country fallback. Populate <code>LOCATION_OVERRIDES</code> with 30 common high-conflict regions: Ukraine, Gaza, Taiwan, Kashmir, etc. Write <code>geocode_location(place_name) → Optional[tuple[float, float, str]]</code> — the third value is the tier used. Test: Ukraine, Gaza, South China Sea, Kashmir. Record which hit each tier. Write tests: <code>assert geocode_location("London")</code> returns coordinates within 0.5° of (51.5, -0.1).
WEEK 6	Write <code>backend/nlp/pipeline.py</code>: <code>process_pending_articles()</code> orchestrator using the 7-step pipeline from Chapter 8.1. Ensure graceful failure at each step: if any step fails, log the <code>error_reason</code> and mark article processed to prevent infinite retries. Implement <code>backend/scheduler.py</code> with <code>APScheduler AsyncIOScheduler</code>: schedule <code>process_pending_articles</code> every 30 seconds. Wire scheduler into FastAPI lifespan context (start on startup, shutdown on shutdown). Test: manually insert a raw article, wait 30s, verify it gets scored, geocoded, and the <code>tier_used</code> logged.
WEEK 7	Implement Python-based NewsAPI fetcher (Rust comes in Phase 2): <code>backend/ingestion/news_fetcher.py</code>. Fetch with query "airspace conflict military aviation" from NewsAPI every 60 seconds. Add RSS ingestion: <code>feedparser</code> for BBC World, Reuters, AP. Schedule every 5 minutes. No API key needed.

Store raw articles via `insert_article()` with `ON CONFLICT DO NOTHING` deduplication on URL.

Run the full system for 2 hours. Verify 100+ articles ingested, processed, and geocoded.

Spot-check 10 processed articles: are geocoding tiers logged? Are sentiment scores plausible?

WEEK 8

Build the evaluation corpus fixture: find 50 articles from the historical events in Chapter 14.3.

Label them manually (`true_positive` / `true_negative` / `hard_negative`) and store in `tests/fixtures/evaluation_corpus/`.

Run `process_pending_articles` against the labelled corpus, compute precision and recall.

Log results to `nlp_evaluation_results` table with `model_name=ProsusAI/finbert` and current thresholds.

Build minimal HTML frontend: article list with sentiment colour badges. Mount as `StaticFiles` in FastAPI.

Write `frontend/js/app.js`: fetch `/api/news/latest` on load, render article cards with filter bar.

WEEK 9

Add Leaflet.js to frontend via CDN. Initialise map with CartoDB Dark Matter tiles.

Add `/api/news/map-data` endpoint: geocoded articles only (lat, lon, headline, score, source).

Fetch `map-data`, add circle markers to map coloured by sentiment severity.

Add popup to each marker: click → article headline, sentiment score, source, time, geocoding tier.

Test at multiple zoom levels. Verify markers are clickable and popups render correctly.

WEEK 10

End-to-end test: start from a clean DB, run for 6 hours, verify all pipeline stages run without intervention.

Fix any bugs found. Prioritise data quality over UI polish at this stage.

Rerun evaluation corpus: compare precision/recall to Week 8 baseline. Document any regression.

Write `docs/phase1-validation.md`: article count, geocoding tier breakdown, model accuracy, known issues.

Commit and tag: `git tag v0.1.0-phase1`.

PHASE 2 · FLIGHT DATA LAYER

Weeks 11–20

✓ Live flights on map, WebSocket real-time updates, Rust ingestion pipeline operational

MILESTONE

Live flights displayed on Leaflet map from OpenSky, refreshed dynamically

Week 13

MILESTONE

WebSocket live updates: flights and alerts update without any page refresh

Week 16

MILESTONE

Rust ingestion pipeline operational, replacing Python news and flight fetchers

Week 20

WEEK 11	Add flight_positions hypertable to DB schema: Alembic migration. Set 90-day retention policy immediately.
	Create SQLAlchemy ORM model for FlightPosition.
	Write backend/api/routes/flights.py: GET /api/flights/live returns latest position per aircraft as GeoJSON.
	Register a free OpenSky account at opensky-network.org to upgrade from 400 to 4,000 req/day.
	Implement Python OpenSky fetcher: backend/ingestion/flight_fetcher.py. Fetch global state at 30s intervals. Bulk INSERT positions with ON CONFLICT (icao24, time) DO NOTHING.
WEEK 12	Add flight ingestion to APScheduler. Verify DB accumulates positions: SELECT COUNT(*) FROM flight_positions.
	Build /api/flights/live: most recent position per icao24 within last 35 seconds (5s grace window).
	Add aircraft to Leaflet map: fetch /api/flights/live, render as small circle markers.
	Add heading-based CSS rotation on SVG aircraft icons.
	Add aircraft popup: callsign, altitude (m → ft), speed (m/s → km/h), origin country, last contact.
WEEK 13	Implement WebSocket endpoint: backend/api/websockets.py with LiveUpdateManager class.
	Add WS /ws/live route to FastAPI.
	Broadcast flight_update message after each successful flight ingestion.
	Write frontend/js/websocket.js: GeoAirWebSocket class with connect(), onmessage, auto-reconnect with exponential backoff.
	Test: open dashboard, verify aircraft positions update every 30s without page refresh. Test reconnect: stop server, restart, verify client reconnects within 30s.
WEEK 14	Overlay news events and flights on the same map simultaneously.
	Add layer toggles: checkboxes to show/hide flights and news events independently.
	Add connection status indicator to top nav bar: green dot if WS connected, red if disconnected.
	Test map at different zoom levels and with both layers active simultaneously.
WEEK 15	Rust fundamentals week: complete The Rust Book chapters 1–6 if not already done.
	Complete tokio.rs/tokio/tutorial chapters 1–4.
	Read PyO3 User Guide: "getting started" and "type conversions" sections.
	Write a minimal async Rust program that fetches a URL and prints the response body. Verify it compiles. This is an intentional lower-pace feature week — invest the time properly in Rust fundamentals.

WEEK 16	Implement <code>rust_core/src/ingestion/news.rs</code> : <code>NewsApiResponse</code> , <code>Article</code> , <code>ArticleSource</code> structs with <code>serde</code> .
	Write <code>fetch_news_api()</code> async function using <code>reqwest</code> + <code>tokio_retry</code> for automatic retries.
	Implement <code>rust_core/src/errors.rs</code> : <code>IngestionError</code> enum using <code>thiserror</code> .
	Add <code>tracing::info!</code> and <code>tracing::warn!</code> log events throughout the ingestion code.
	Write unit tests in <code>news.rs</code> : <code>Article</code> deserialisation from known JSON string. Run cargo test.
	Write <code>rust_core/src/lib.rs</code> : <code>PyO3</code> module definition. Expose <code>fetch_news</code> as <code>#[pyfunction]</code> .
	Build with <code>maturin develop</code> . Test from Python REPL: <code>import rust_core; print(rust_core.fetch_news(key, "test"))</code> .
WEEK 17	Implement <code>rust_core/src/ingestion/flights.rs</code> : <code>OpenSky</code> response structs, <code>fetch_opensky_bbox()</code> with retry logic.
	Expose <code>fetch_flights(lat_min, lon_min, lat_max, lon_max)</code> as <code>#[pyfunction]</code> .
	Build and test: verify <code>fetch_flights</code> returns a list of dicts from Python.
	Compare output of Rust <code>fetch_flights</code> vs Python <code>flight_fetcher</code> on the same bounding box field-for-field.
	If fields differ, fix <code>serde</code> mapping — the Rust implementation must be a drop-in replacement.
WEEK 18	Implement <code>rust_core/src/db.rs</code> : <code>create_pool()</code> and <code>store_articles()</code> using <code>sqlx</code> .
	Test direct DB writes from Rust: insert 5 test articles, verify in <code>psql</code> .
	Add <code>store_flights()</code> to <code>db.rs</code> : bulk insert flight positions.
	Build a standalone Rust binary (add <code>[[bin]]</code> to <code>Cargo.toml</code>) running news + flight ingestion in a loop.
	Test the binary for 30 minutes: verify counts accumulate at expected rate.
WEEK 19	Replace Python <code>news_fetcher.py</code> calls with <code>rust_core.fetch_news()</code> in the scheduler.
	Replace Python <code>flight_fetcher.py</code> calls with <code>rust_core.fetch_flights()</code> in the scheduler.
	Run Rust-powered system for 4 hours. Compare article and flight counts vs Python baseline.
	Keep Python fetchers intact but disabled — they are the fallback if Rust encounters issues.
WEEK 20	Run Rust-powered system for 48 hours continuously. Profile memory and CPU with <code>htop</code> .
	Fix any panics or errors observed in tracing logs during the 48-hour run.
	Write <code>tests/python/test_rust_bridge.py</code> : verify <code>rust_core</code> functions return correct types and expected keys.
	Commit and tag: <code>git tag v0.2.0-phase2</code> . Review Milestones 2A, 2B, 2C.

PHASE 3 · CORRELATION ENGINE V1

Weeks 21–32

✓ End-to-end correlation alerts, confidence scoring, map overlays, calibration below 25% FP rate

MILESTONE	Correlation alerts appearing in DB for real geopolitical events with evaluation corpus passing	Week 24
MILESTONE	Alerts displayed in UI with affected zone overlays on map and acknowledge flow working	Week 27
MILESTONE	Confidence scoring calibrated below 25% false positive rate on 50-alert review	Week 32
WEEK 21	Implement backend/geo/spatial.py: haversine_km(), get_bounding_box(), point_in_circle(). Write unit tests: haversine_km("London","Paris") should return 340-350km within 5% tolerance. Add correlations table to DB schema: Alembic migration. Include algorithm_version and affected_flights JSON fields. Create SQLAlchemy ORM model for Correlation.	
WEEK 22	Implement backend/correlation/engine.py: correlate_article() function using bounding circle spatial matching. Implement backend/correlation/scoring.py: calculate_confidence() using the 5-factor formula from Chapter 9.2. Implement classify_alert() using thresholds from Chapter 9.3. Wire correlation engine to APScheduler: run run_correlation_pass() every 45 seconds. run_correlation_pass(): fetch recently geocoded articles, fetch live flights, call correlate_article() for each, store results.	
WEEK 23	Observe first correlations: SELECT * FROM correlations ORDER BY detected_at DESC LIMIT 10. Manually review each: is the correlation geopolitically plausible? Is the event actually significant? Start the false-positive tracking spreadsheet: article headline, detected region, flight count, confidence, correct? Run evaluation corpus against correlation engine. Record precision/recall in nlp_evaluation_results. Identify patterns in false positives — common failure modes: wrong geocoding, low-relevance articles scoring too high.	
WEEK 24	Extend WebSocket to broadcast new_alert events for correlations with level >= HIGH. Implement frontend alert panel: right-column div showing alert cards sorted by confidence. Alert card: region name, alert level badge (colour-coded), flight count, headline, time, confidence score. Handle new_alert in frontend/js/alerts.js: prepend new card, animate entry. Test: insert a test article manually, verify alert appears in browser within 60 seconds.	

WEEK 25	Add event zone overlay to Leaflet map: semi-transparent circle per active correlation. Circle radius corresponds to the correlation radius_km field. Add CSS pulsing animation on active/critical alert zone circles. Colour aircraft icons inside an active event zone red; others remain default colour. Click event zone: popup shows headline, confidence score, flight count, detected time, alert level.
WEEK 26	Add GET /api/correlations/active endpoint: unacknowledged correlations sorted by confidence desc. Add POST /api/alerts/{id}/acknowledge: marks acknowledged=true, sets acknowledged_at timestamp. Add Acknowledge button to each alert card. Test end-to-end: click → card disappears → DB updated. Add GET /api/correlations/history: all correlations for a region over configurable time window.
WEEK 27	Calibration sprint: analyse 2+ weeks of false positive data from the tracking spreadsheet. Tune confidence weights against the evaluation corpus: adjust factors until evaluation F1 > 0.70. Add source credibility scoring: maintain CREDIBLE_SOURCES dict mapping domains to credibility scores. Test calibrated weights on the evaluation corpus — verify improvement over Week 24 baseline. Target: evaluation corpus F1 > 0.70 and false positive rate < 35% (working toward 25% final target).
WEEK 28	Implement backend/correlation/projection.py: project_flight_position() extrapolating from current heading and speed. Implement will_enter_zone(flight, event, hours=2): returns bool if flight projected to enter zone. Add projected=true flag to affected_flights JSON for aircraft headed toward the zone. Draw projected flight path lines on map as dashed lines (current position → projected entry point).
WEEK 29	Add 7-day sentiment sparklines per monitored region using Chart.js. Create /api/news/sentiment?region=X endpoint: daily average sentiment scores. Add a history tab to the UI: list all correlations for a selected region over 30 days. Add a summary stats row to the dashboard: total alerts this week, active alerts, regions monitored.
WEEK 30	Historical validation (Chapter 14.3): insert archived news articles from Russia-Ukraine Feb 2022 into DB. Run correlation engine against them. Verify high-confidence alerts for Eastern European airspace. Repeat for Gaza Oct 2023 and Pakistan-India Feb 2019. Document results in docs/historical-validation.md: what was detected, what was missed, confidence vs actual severity.

WEEK 31	Performance profiling: measure NLP pipeline throughput (articles per minute).
	If below 20 articles/minute, implement batch inference: collect 16 articles, pass to pipeline in one call.
	Measure WebSocket latency: time from correlation insert to browser notification. Target < 5 seconds.
	Profile slowest API endpoints with py-spy or cProfile.

WEEK 32	Full false-positive audit: review 50 most recent correlations, categorise as TP/FP/borderline.
	If false positive rate still above 25%, raise the minimum confidence threshold from 0.35 to 0.45.
	Write tests/python/test_correlation.py: comprehensive unit and integration test suite.
	Run evaluation corpus one final time: log results. Must show improvement from Week 23 baseline.
	Commit and tag: git tag v0.3.0-phase3. Write phase3-validation.md.

PHASE 4 · HARDENING & DEPLOYMENT

Weeks 33–44

✓ Cloud-deployed, monitored, documented, 7-day stability — platform in battle-ready state

MILESTONE	Platform running on cloud server, accessible via HTTPS public URL	Week 37
MILESTONE	Full documentation complete, environment reproducible on a fresh machine from README only	Week 40
MILESTONE	Platform runs 7 days unattended, monitoring alerts configured, zero manual interventions	Week 44

WEEK 33	Implement JWT authentication: POST /api/auth/register, POST /api/auth/token (returns JWT).
	Add Depends(get_current_user) to all protected endpoints.
	Add login form to frontend. Store JWT in localStorage, attach as Authorization: Bearer header.
	Test: register, obtain token, access protected route, verify 401 without token.

WEEK 34	Write docker-compose.yml with all services: db (TimescaleDB), redis, backend, nginx.
	Write backend/Dockerfile: FROM python:3.11-slim, install deps, build Rust module with maturin, start uvicorn.
	Test Docker Compose locally: docker compose up — verify all services start and communicate.
	Common issues: Rust build fails in container (add build-essential, libssl-dev to Dockerfile), DB not ready (add healthcheck dependency).
	Run Dockerised system for 2 hours locally — verify ingestion and correlation continue working.

WEEK 35	Add /api/health endpoint: returns DB connection, last ingestion timestamp, article count in last hour, WebSocket connection count.
	Write nginx.conf: proxy /api/* and /ws to backend, serve frontend static files from /usr/share/nginx/html.
	Configure nginx rate limiting: limit_req_zone to protect against traffic spikes.
	Test nginx routing locally. Verify /api/health returns correctly through nginx.
WEEK 36	Provision Hetzner CX22 server (~£4/month): 2 vCPU, 4GB RAM, 40GB SSD.
	SSH setup: generate key pair, add to Hetzner, disable password auth in /etc/ssh/sshd_config.
	Configure UFW firewall: allow only ports 22 (SSH), 80 (HTTP), 443 (HTTPS).
	Install Docker on server following docker.com official Ubuntu 24.04 guide.
	Clone repository to server via GitHub. Create .env on server manually — never commit .env.
WEEK 37	Register domain name (~£10/year). Point A record at server IP.
	Configure HTTPS with Let's Encrypt via certbot Docker container.
	Run docker compose up -d on server. Verify platform accessible via domain in browser.
	Test all critical paths in production: ingestion, flight display, correlations, WebSocket, authentication.
	Share URL with a non-developer — verify it works for someone unfamiliar with the codebase.
WEEK 38	Set up UptimeRobot (free tier): HTTP monitor on /api/health. Email alert if down > 5 minutes.
	Enable PostgreSQL slow query logging: log_min_duration_statement = 500ms in postgresql.conf.
	Set up log rotation on server: logrotate for Docker container logs.
	Add HTTPS redirect in nginx: all HTTP requests permanently redirect to HTTPS (301).
WEEK 39	Write comprehensive README.md: project overview, prerequisites, environment setup, run, deploy.
	Write docs/architecture.md: component diagram, data flow description, technology decisions.
	Write docs/correlation-algorithm.md: confidence formula, weights, version history, calibration log.
	Write docs/api-reference.md: every endpoint, parameters, response format, example payloads.
	Test: ask someone unfamiliar with the project to set it up using only the README. Fix any gaps.
WEEK 40	Full code review pass: every file in backend/. Fix missing error handling, hardcoded values, missing logging.
	Move all remaining hardcoded values (thresholds, credible sources, model names) into config.py.
	Audit all SQL: every query must use parameterised inputs. Zero f-string SQL.
	Remove all print() debug statements. Replace with structured tracing/logging calls.

WEEK 41	7-day stability test begins: platform running on Hetzner, no manual intervention allowed.
	Monitor UptimeRobot daily. Note any downtime events with timestamps.
	Check DB size growth daily: <code>SELECT pg_database_size('geoair')</code> . Verify retention policy is working.
	Monitor memory: <code>free -h</code> on server daily. Watch for any OOM events.
WEEK 42	Address any issues found during the 7-day stability test.
	Set up automated weekly DB backups: <code>pg_dump geoair gzip > backup_\$(date +%Y%m%d).gz</code> .
	Test the restore procedure: restore from backup to a test DB, verify data integrity.
	Commit and tag: <code>git tag v0.4.0-phase4</code> .
WEEK 43	Security audit: review all API endpoints for missing auth, check for information leakage in error messages.
	Add slowapi rate limiting: 100 requests/minute per IP. Test with ab or locust.
	Review Docker security: containers run as non-root user, no unnecessary capabilities.
	Add CORS policy: restrict to known frontend domain in production, wildcard only in development.
WEEK 44	Personal reflection: write docs/phase4-retrospective.md — what worked, what was hard, what Phase 5 will tackle.
	Platform checkpoint: stable, deployed, documented, secured, monitored. Everything in Git.

PHASE 5 · CORRELATION ENGINE V2 + REACT FRONTEND

Weeks 45–56

✓ Polygon spatial matching, GDELT integration, React migration, < 20% false positive rate

MILESTONE	Correlation engine v2 with FIR polygon matching and Goldstein scale scoring operational	Week 48
MILESTONE	React frontend migration complete with all features working identically to vanilla JS version	Week 52
MILESTONE	GDELT integration live, false positive rate < 20% on evaluation corpus	Week 56
WEEK 45	Review Phase 3–4 calibration data: what were the most common false positive patterns?	
	Design v2 algorithm: replace bounding circle with PostGIS ST_Within for FIR polygon matching.	
	Download OurAirports data: ourairports.com/data — FIR boundary polygons.	
	Parse and import FIR boundary polygons into airspace_regions table. Add PostGIS GIST index.	
WEEK 46	Implement v2 spatial correlation: find containing FIR polygon for each article location using PostGIS ST_Within.	

	Replace Python shapely calculations with PostGIS for all flight-in-zone queries — significant performance improvement.
	Set algorithm_version="v2" on new correlations. Run v1 and v2 in parallel for comparison.
WEEK 47	Implement GDELT integration in Rust: fetch_latest_gdelt_events() — downloads 15-minute bulk CSV.
	Parse GDELT CSV: event_code, goldstein_scale, action_lat, action_lon, source_url.
	Map Goldstein scale (-10 to +10) to sentiment_score (-1.0 to 1.0).
	Store GDELT events in articles table with source_name="GDELT" for distinguishable filtering.
WEEK 48	Integrate Goldstein scale into confidence scoring: use as an additional weight factor.
	GDELT events with Goldstein < -5 (highly conflictual) should boost correlation confidence directly.
	Run v1 and v2 comparison: do v2 alerts have lower false positive rates for the same real events?
	Run evaluation corpus against v2. Must show F1 improvement over v1 baseline.
WEEK 49	Begin React migration: npm create vite@latest frontend-react -- --template react.
	Recreate news feed panel as React component: fetch /api/news/latest, render article cards with live state.
	Recreate alert panel as React component with WebSocket integration via useEffect and useReducer.
	Run both vanilla JS and React frontends simultaneously to verify feature parity as you migrate.
WEEK 50	Migrate Leaflet map to React using react-leaflet.
	Recreate flight layer, event markers, and zone overlays as React components.
	State management: useReducer for flight positions (updated every 30s by WebSocket).
	Performance test: verify React map updates without excessive re-renders. Use React DevTools Profiler.
	Fix re-render issues: React.memo on aircraft markers, useMemo for flight position transforms.
WEEK 51	Complete remaining React components: header bar, filter controls, sparklines (react-chartjs-2).
	Implement full responsive CSS Grid layout in React.
	Mobile layout: map collapses to full width on screens < 768px, panels become vertical tabs.
	Test React frontend on a real mobile device.
WEEK 52	Switch production to serve the React frontend from Docker build (npm run build in Dockerfile).
	Remove old vanilla JS frontend (archive in a branch for reference).
	Full end-to-end test of React frontend in production over 24 hours.
	Commit and document the migration in CHANGELOG.md.

WEEK 53	Implement user watchlist: users can save specific regions/countries to a personal watchlist.
	Persist watchlist in user_watchlist table. Filter dashboard to show only watched regions.
	Add advanced analytics page: correlation count over time by region (Chart.js line chart).
	Add Leaflet heatmap layer: frequency of correlations per geographic grid cell.

WEEK 54	Advanced GDELT: add RSS fallback sources (BBC, Reuters, AP, Al Jazeera) to supplement NewsAPI quota.
	Implement feedparser-based RSS pipeline alongside GDELT — ensures real-time coverage without API cost.
	Multilingual news: test spaCy's xx_ent_wiki_sm model for non-English entity extraction on GDELT articles.

WEEK 55	Phase 5 calibration: compile false positive rate from v2 algorithm over the full phase.
	Run evaluation corpus against v2. Target: F1 > 0.80, false positive rate < 20%.
	Update docs/correlation-algorithm.md with v2 weights, calibration history, and v1 vs v2 comparison.
	Commit and tag: git tag v0.5.0-phase5.

WEEK 56	Phase 5 review: compare all milestones against actual delivery. Document any gaps.
	Write docs/phase5-retrospective.md.
	Ensure all tests pass on the React + v2 correlation stack before proceeding.

PHASE 6 · PRODUCTION HARDENING & PUBLIC LAUNCH

Weeks 57–72

✓ 14-day uptime achieved, Grafana monitoring, sub-2s load time, public launch

MILESTONE	Prometheus + Grafana monitoring stack live, 14-day unattended uptime achieved	Week 62
MILESTONE	Performance optimisation complete — dashboard loads in under 2 seconds on standard broadband	Week 67
MILESTONE	Public launch: platform accessible, suitable for non-developer users, v3 correlation engine operational	Week 72

WEEK 57	Add Prometheus metrics endpoint to FastAPI: install prometheus-fastapi-instrumentator.
	Instrument: ingestion_rate, correlation_rate, nlp_throughput, api_latency_p95, websocket_connections.
	Install Grafana as a Docker container. Connect to Prometheus data source.
	Create Grafana dashboard: all 5 metrics above, DB size, and active alert count.

WEEK 58	Implement application-level alerting: email notification if ingestion fails for > 10 minutes.
	Add circuit breaker for NewsAPI: pause requests for 5 minutes after 3 consecutive 5xx responses.
	Add structured logging with correlation IDs: every API request gets a unique X-Request-ID header.
	Ensure all log events include request_id for tracing across ingestion, NLP, and correlation.
WEEK 59	Database query optimisation: EXPLAIN ANALYZE on the 5 most-executed queries.
	Add any missing indexes identified by EXPLAIN output.
	Implement Redis caching for expensive, slow-changing queries: regional sentiment aggregates, correlation history.
	Measure query time improvements before/after. Document in docs/performance-log.md.
WEEK 60	Correlation engine v3: add per-region historical baseline volatility to confidence scoring.
	A region that generates frequent low-severity events (Middle East) should have higher thresholds than a quiet region.
	Add recency decay curve instead of linear: exponential decay after 6 hours, floor at 0.1.
	Tag all new correlations algorithm_version="v3". Run three-way comparison: v1, v2, v3 on evaluation corpus.
WEEK 61	14-day unattended stability test begins.
	Monitor UptimeRobot and Grafana dashboards daily. No manual intervention in the system.
	Document every incident in docs/incident-log.md: what happened, when, how it resolved (or self-healed).
WEEK 62	Address any incidents from the 14-day test.
	Verify Milestone 6A: 14-day uptime achieved.
	Write a brief incident analysis for any downtime events — identify root cause and mitigation.
WEEK 63	Performance sprint: target dashboard load < 2 seconds.
	Implement HTTP caching headers on static assets: Cache-Control: max-age=86400 for JS/CSS bundles.
	Lazy-load Chart.js sparklines: only render when panel is visible (IntersectionObserver).
	Profile React bundle: use Vite's rollup-plugin-visualizer to identify large dependencies.
	Code-split heavy dependencies (leaflet, chart.js) as dynamic imports.
WEEK 64	Email notification system: users can set minimum alert level for email notifications.
	Implement via Python smtplib or SendGrid free tier.
	Test: trigger a CRITICAL correlation, verify email received within 5 minutes.

	Add feedback button to UI: links to Google Form for user feedback.
WEEK 65	Build public-facing landing page explaining GeoAir for non-technical visitors.
	Add onboarding tour: overlay on first visit explaining map layers and alert panel.
	Accessibility review: check WCAG 2.1 colour contrast ratios (minimum 4.5:1 for normal text).
	Keyboard navigation: verify all interactive elements are reachable via Tab key.
WEEK 66	Identify and build 3 historical demo scenarios using real archived data:
	Demo 1: Russia-Ukraine airspace closure Feb 2022 — replay the correlation the engine would have generated.
	Demo 2: A Middle East conflict event — show GeoAir alert with real GDELT article data.
	Demo 3: A South-East Asia territorial dispute — show projected flight path diversion.
WEEK 67	User acceptance testing: ask 3 non-developer users to use the dashboard for 30 minutes.
	Record observations: what was confusing, what was missing, what worked well.
	Implement the top 3 UI improvements from feedback sessions.
	Verify Milestone 6B: dashboard load < 2 seconds from cold cache.
WEEK 68	Write technical blog post: "How GeoAir correlates geopolitical events with live flight data."
	This doubles as portfolio content and documentation. Post on dev.to, personal site, or GitHub README.
	Add GET /api/correlations/export?format=csv for power users.
	Enable Swagger UI at /docs in production: FastAPI generates it automatically.
WEEK 69	Final Docker hardening: set resource limits on all containers in docker-compose.yml (mem_limit, cpus).
	Set up automated weekly DB backups to Backblaze B2 or equivalent remote storage.
	Test restore procedure: restore to a test DB, verify data integrity and system restarts cleanly.
WEEK 70	Final documentation pass: README, architecture, API reference, correlation algorithm, all up to date.
	Create CHANGELOG.md documenting all major features and fixes across all 6 phases.
	Add VERSION file to project root: 1.0.0.
WEEK 71	Soft launch: share URL with small trusted group (friends, aviation communities).
	Monitor for unexpected traffic patterns or errors during soft launch week.
	Respond to any feedback received.

WEEK 72	Public launch: share on r/aviation, r/geopolitics, Hacker News Show HN, relevant Discord servers.
	Write personal retrospective: docs/retrospective.md — what you built, what you learned, what you'd do differently.
	Commit and tag: git tag v1.0.0.

CHAPTER 14

Testing Strategy

14. Testing Strategy

Testing is the mechanism that lets you change the correlation engine confidently. Without a test suite, every algorithm change risks breaking something that was working. The guiding rule: write tests before you start the next phase, not after.

14.1 Testing Pyramid

Level	Count Target	Runs When	Covers
Unit tests (Python)	40–60	Every commit	Individual functions: geocoding, scoring, spatial, NLP
Unit tests (Rust)	15–25	cargo test	Serde parsing, HTTP client, error types, retry logic
Integration tests	15–20	Before each phase tag	API endpoints, DB writes, NLP pipeline end-to-end
Evaluation corpus runs	Continuous	On model/threshold change	Precision, recall, F1 per NLP model version — Chapter 8.2
Historical validation	5–10 events	Monthly	Correlation engine against known real-world events
Stability test	1 per phase	Phase end	7-day (Phase 4) and 14-day (Phase 6) unattended operation

14.2 Key Unit Tests to Write

Python — Spatial and Correlation

- `test_haversine_london_paris`: `haversine_km(51.5, -0.1, 48.9, 2.4)` should return 340–350km.
- `test_confidence_high_severity`: `calculate_confidence(-0.85, 45, credible_source, model_conf=0.92)` should return ≥ 0.72 .
- `test_confidence_penalised_low_model_confidence`: `calculate_confidence(-0.85, 45, credible_source, model_conf=0.52)` should return < 0.60 — the new `model_confidence` factor penalises uncertain predictions.
- `test_confidence_low_severity`: `calculate_confidence(-0.20, 2, unknown_source, model_conf=0.70)` should return < 0.35 .
- `test_correlate_article_no_location`: `correlate_article({"sentiment_score": -0.8, "lat": None})` should return `None`.
- `test_correlate_article_positive_sentiment`: articles with score > -0.2 should never produce a correlation.

Python — NLP

- `test_entity_extraction_ukraine`: `extract_geopolitical_entities("Russian forces advanced into Ukraine...")` should return "Ukraine" and "Russia" as GPE entities, not LOC.
- `test_sentiment_negative_finbert`: `score_sentiment("Escalating violence and airspace closures...")` should return score < -0.3 with confidence > 0.7.
- `test_sentiment_neutral`: `score_sentiment("The minister held routine talks today...")` should return score between -0.2 and 0.2.
- `test_geocode_redis_cache`: call `geocode_location("Kyiv")` twice. Second call should hit Redis tier (verify with cache stats).
- `test_geocode_override`: `geocode_location("Gaza Strip")` should return coordinates from `LOCATION_OVERRIDES` without calling Nominatim.

Rust — Serde and Ingestion

- `test_article_deserialise`: parse known JSON string into Article struct, verify all fields.
- `test_article_missing_content`: article JSON without "content" field should produce Article with content = None.
- `test_flight_deserialise`: parse OpenSky response row into FlightState struct, verify numeric fields.
- `test_retry_logic`: mock HTTP server returns 503 twice then 200. Verify `fetch_news_api()` returns successfully after retries.

14.3 Historical Validation Events

Event	Date	Expected Behaviour	How to Test
Russia-Ukraine invasion begins	Feb 24 2022	High-confidence CRITICAL alerts for Eastern European airspace, mass rerouting signals	Import archived BBC/Reuters articles from that date
Sudan conflict escalation	Apr 15 2023	Alerts for Khartoum FIR, Sudanese airspace closures	Import AP articles from Apr 15-17 2023
Israel-Gaza conflict begins	Oct 7 2023	Alerts for Israeli/Jordanian airspace, airline diversions	Import articles from Oct 7-10 2023
Pakistan-India standoff	Feb 27 2019	Alerts for Pakistani airspace closure, India-Europe diversions	Import articles from Feb 27-28 2019
Myanmar coup	Feb 1 2021	Moderate alerts for Southeast Asian airspace	Validate that low-aviation-impact events produce MEDIUM not CRITICAL alerts

◆ TIP

Store all historical test articles as JSON fixtures in `tests/fixtures/historical/`. Running the full historical validation suite should be a single pytest command — no manual article insertion required.

CHAPTER 15

Deployment & DevOps

15. Deployment & DevOps

15.1 Cloud Provider Selection

Provider	Monthly Cost	Specs	Notes
Hetzner CX22	~£4	2 vCPU, 4GB RAM, 40GB SSD, 20TB bandwidth	RECOMMENDED — best value in Europe. Upgrade to CX32 (8GB, ~£8) if memory issues arise.
DigitalOcean Basic	~£8	1 vCPU, 2GB RAM, 50GB SSD	Excellent documentation. 2GB RAM is too tight for full NLP stack.
Vultr High Frequency	~£5	1 vCPU, 2GB RAM, 64GB NVMe	Fast NVMe. Same 2GB RAM caveat as DigitalOcean.
Oracle Cloud Free	£0	Up to 4 vCPU, 24GB RAM	Free but limited egress and complex networking. HuggingFace model needs ~1.5GB RAM alone.

◆ WARNING

4GB RAM is the minimum for running PostgreSQL + TimescaleDB + Redis + FastAPI + the HuggingFace NLP model simultaneously. The NLP model alone uses approximately 1.5GB when loaded. Start with Hetzner CX22 — upgrade to CX32 (8GB, ~£8/month) if memory becomes an issue in Phase 5.

15.2 Monitoring Stack

Tool	Cost	Purpose
UptimeRobot	Free	HTTP uptime monitoring, email alerts on downtime. Set up in Phase 4.
/api/health endpoint	Built-in	Returns DB status, last ingestion time, article count in last hour, WS connections.
Prometheus + Grafana	Free (Phase 6)	Full metrics dashboard. See Chapter 15.3 for metric definitions.
Docker logs	Built-in	docker compose logs -f backend — tail live application logs.
pg_activity	Free	Real-time PostgreSQL query monitoring. Detect slow queries in production.
logrotate	Built-in	Prevent Docker log files from filling the disk. Configure in Phase 4.

15.3 Key Prometheus Metrics

Metric Name	Type	What It Tracks
geoair_articles_ingested_total	Counter	Total articles ingested, labelled by source (newsapi, gdelt, rss)
geoair_nlp_processing_seconds	Histogram	Time to process each article through the full NLP pipeline
geoair_correlation_alerts_total	Counter	Total alerts generated, labelled by level (critical, high, medium)
geoair_ingestion_lag_seconds	Gauge	Seconds since last successful ingestion — alert if > 300
geoair_websocket_connections	Gauge	Current number of active WebSocket connections
geoair_geocoding_tier	Counter	Geocoding calls per tier — reveals cache hit rate

CHAPTER 16

Risks & Mitigation

16. Risks & Mitigation

HIGH · NLP domain mismatch produces systematic false positives

Sentiment models trained on tweets or general text classify geopolitical events differently from domain-tuned models. "Ceasefire declared" reads as positive in a general model but may indicate ongoing active conflict. Mitigation: start with ProsusAI/finbert (geopolitical domain) not a Twitter model. Build the evaluation corpus in Week 8 and run it on every model change. Plan the DeBERTa fine-tuning track in Phase 5 if finbert accuracy is insufficient.

HIGH · Rust learning curve stalls Phase 2

PyO3 and async Rust are genuinely hard. The ownership model, lifetimes, and async runtime require sustained study before they click. Mitigation: implement Python fallback fetchers in parallel with Rust work (already built in Phase 1). If Rust is not working by Week 17, deploy Python fetchers to production and continue Rust as a parallel workstream. Never block the critical path on the Rust component.

HIGH · Correlation engine loses momentum in weeks 21-32

The engine is unglamorous to debug and takes weeks before it produces impressive results. False positive rates above 50% in early weeks are demoralising but normal. Mitigation: track everything in the false-positive spreadsheet from day one of Phase 3 — measurable progress is motivating. Set a two-week rule: if stuck on any single problem for two weeks, simplify the algorithm rather than the ambition. A working v1 is always better than a stuck v2.

HIGH · OpenSky rate limit exceeded in production

At 10-second global polling, the system requires 8,640 requests/day against a 4,000/day limit for registered accounts. Mitigation: global polling at 30 seconds (2,880 req/day). Increase to 10 seconds dynamically only for bounding boxes containing active high-severity alerts. Add ADS-B Exchange as a supplementary source. This is a design decision, not a workaround — the dynamic polling rate is a feature of the architecture.

MEDIUM · Geocoding volume hits Nominatim rate limit

At 1 request/second and 30-second processing intervals, the geocoding pipeline can fall behind on article backlogs. Mitigation: the tiered geocoding architecture in Chapter 5.1 solves this. The Redis cache means most requests never hit Nominatim. The LOCATION_OVERRIDES dictionary handles the top 30 high-frequency conflict regions at sub-millisecond latency. Monitor the geocoding_tier metric in Grafana to verify cache hit rates.

MEDIUM · HuggingFace NLP model too slow on CPU

finbert inference can take 300-800ms per article on CPU. At 30-second pipeline runs processing large backlogs, this becomes a bottleneck. Mitigation: implement batch inference in Phase 3 — process 16 articles simultaneously (much faster per article). distilbert is 3x faster as a development fallback. GPU instance upgrade is an option if production throughput demands it (Hetzner GPU servers available from ~£30/month).

MEDIUM · React migration introduces regressions in Phase 5

Migrating a real-time WebSocket app with Leaflet to React is non-trivial. High-frequency WebSocket updates can cause excessive re-renders that make the map unusable. Mitigation: run both frontends in parallel during Weeks

49-52. Use React.memo on aircraft markers, useReducer for flight state, and React DevTools Profiler to identify render bottlenecks before switching production. Only switch after every feature is verified to work identically.

LOW · TimescaleDB disk usage exceeds server capacity

Flight positions at 30-second global intervals generate 200-400MB per day. Mitigation: set the 90-day retention policy in Week 11 (Phase 2), not Phase 4. Only store flights within relevant bounding boxes when possible. Monitor disk weekly with `pg_database_size()`.

CHAPTER 17

Glossary

17. Glossary

Term	Definition
ADS-B	Automatic Dependent Surveillance–Broadcast. Technology used by aircraft to broadcast their GPS position, altitude, speed, and identity to ground receivers.
APScheduler	Advanced Python Scheduler. Library for running recurring background jobs within a Python application.
Async / Await	Programming pattern for concurrent code. An async function can pause while waiting for I/O, allowing other tasks to run on the same thread.
CAMEO	Conflict and Mediation Event Observations. A standardised coding system used by GDELT to categorise political events into 300+ event types.
Confidence Score	A 0.0–1.0 score representing GeoAir's certainty that a correlation between a news event and aviation impact is meaningful.
Correlation Engine	The core algorithm that matches geocoded news events to aircraft positions in potentially affected airspace.
DeBERTa	A state-of-the-art transformer NLP model. Planned for fine-tuning on geopolitical text in Phase 5 to improve domain accuracy.
Docker / Docker Compose	Containerisation tools. Docker packages applications with dependencies. Compose orchestrates multiple containers as a system.
Evaluation Corpus	A labelled dataset of 250 articles used to measure NLP model precision, recall, and F1 score — built in Week 8.
F1 Score	Harmonic mean of precision and recall. A single metric that balances both. Target F1 > 0.70 for correlation engine v1.
FastAPI	A modern Python web framework. Known for automatic OpenAPI documentation generation and native async/await support.
FIR	Flight Information Region. An airspace designation defining the boundaries within which a particular air traffic service is responsible.
finbert	ProsusAI/finbert. A BERT model fine-tuned on financial and geopolitical news text. Recommended starting NLP model for GeoAir.
GDELT	Global Database of Events, Language, and Tone. A free, unlimited global news database updated every 15 minutes with Goldstein scale event scoring.
Geocoding	Converting a place name string (e.g. "Kyiv") into geographic coordinates (latitude, longitude).
Goldstein Scale	A numerical scale in GDELT ranging from -10 (most conflictual) to +10 (most cooperative) used to characterise events.
Hypertable	A TimescaleDB concept. A regular PostgreSQL table automatically partitioned by time for efficient time-series queries.

Term	Definition
ICAO24	A unique 24-bit identifier assigned to each aircraft, expressed as a 6-character hexadecimal string.
JWT	JSON Web Token. A compact, URL-safe means of representing claims for authentication between client and server.
maturin	A build tool for PyO3 Rust extensions. Compiles Rust code and installs the resulting .so module into the Python virtual environment.
NLP	Natural Language Processing. Computational techniques for analysing and understanding human language text.
OpenSky Network	A free, open-source network of ADS-B receivers providing live flight tracking data via a REST API.
PostGIS	A PostgreSQL extension adding spatial/geographic data types and functions. Used for FIR polygon queries in Phase 5.
Precision	Of all alerts generated, what fraction were correct. High precision = few false positives.
PyO3	A Rust library enabling Rust code to be compiled into Python extension modules via the C extension API.
Recall	Of all real events, what fraction were detected. High recall = few false negatives (missed events).
Redis	An in-memory key-value store. Used as the geocoding cache tier and optionally as a message queue.
Sentiment Analysis	NLP technique for identifying emotional tone of text (positive, negative, neutral) and quantifying it as a score.
shapely	A Python library for geometric operations on 2D shapes. Used for spatial calculations before PostGIS migration in Phase 5.
spaCy	A Python NLP library specialising in named entity recognition — identifying people, places, and organisations in text.
TimescaleDB	A PostgreSQL extension adding time-series capabilities including automatic time partitioning and retention policies.
tokio	Rust's primary async runtime. Provides the event loop, task scheduler, and async I/O primitives.
WebSocket	A communication protocol providing full-duplex communication between browser and server over a single persistent connection.

CHAPTER 18

Resources & Further Reading

18. Resources & Further Reading

Category	Resource	URL	When to Use
Rust	The Rust Book	doc.rust-lang.org/book	Primary learning resource. Complete chapters 1–12 before starting Phase 2. Ownership and borrowing are essential.
Rust	Tokio Tutorial	tokio.rs/tokio/tutorial	Essential before writing any async Rust. Complete before touching the ingestion pipeline.
Rust	PyO3 User Guide	pyo3.rs	Complete reference for Rust/Python interop. Read "getting started" and "type conversions" in full.
Rust	Zero To Production In Rust	zerotomastery.io	Best book for real-world Rust web services. Optional but highly recommended.
Python	FastAPI documentation	fastapi.tiangolo.com	Excellent official docs. Start with the tutorial, then WebSocket and dependency injection sections.
Python	SQLAlchemy 2.0 async	docs.sqlalchemy.org	Read the async session usage section specifically. v2.0 syntax differs significantly from v1.x.
NLP	ProsusAI/finbert model card	huggingface.co/ProsusAI/finbert	Recommended starting sentiment model. Check the model card for usage examples and accuracy benchmarks.
NLP	HuggingFace Course	huggingface.co/learn/nlp-course	Free, comprehensive introduction to transformers and the HuggingFace ecosystem.
NLP	spaCy 101	spacy.io/usage/spacy-101	Read before writing any NLP code. Covers Doc, Token, Span, and entity types you will use throughout.
NLP	DeBERTa paper	arxiv.org/abs/2006.03654	Background on the DeBERTa architecture planned for Phase 5 fine-tuning.
Database	TimescaleDB docs	docs.timescale.com	Hypertables, retention policies, continuous aggregates — all three are used in GeoAir.

Category	Resource	URL	When to Use
Database	PostGIS documentation	postgis.net/documentation	Spatial functions for Phase 5 FIR polygon queries: ST_Within, ST_DWithin, ST_GeomFromText.
Database	Use The Index, Luke	use-the-index-luke.com	Best practical guide to SQL indexing. Read before writing complex queries.
Data Sources	OpenSky Network API	opensky-network.org/apidoc	Full REST API reference. Focus on /states/all endpoint with bounding box parameters.
Data Sources	GDELT documentation	gdeltproject.org/data.html	GDELT 2.0 data format docs. Focus on the export.CSV format and Goldstein scale description.
Data Sources	OurAirports data	ourairports.com/data	FIR boundary polygon data. Download airports.csv and boundaries.csv for Phase 5.
Geospatial	Leaflet.js documentation	leafletjs.com/reference.html	TileLayer, Marker, CircleMarker, Polyline, and Popup are the core classes used in GeoAir.
Deployment	Docker Compose reference	docs.docker.com/compose	Complete docker-compose.yml syntax reference.
Deployment	Prometheus docs	prometheus.io/docs	Metrics collection and alerting configuration for Phase 6.
Deployment	Grafana dashboards	grafana.com/docs	Dashboard creation for the monitoring stack.

GeoAir Intelligence Platform · Master Project Plan v3.0

Jack · 2025–2027 · Private & Confidential

This document is a living guide. Update it as the project evolves — especially the correlation algorithm version history, confidence scoring weights, and deployment configuration. The best version of this document is the one that reflects what you actually built.